Graphic visualization of data

just scratch the surface

Learning Objectives

- Understand plotting paradigms in R
- Create base R plots
- Use ggplot2 grammar of graphics
- Customize and export plots

Why Visualization?

- Exploratory Data Analysis (EDA)
- Communicating results
- Detecting patterns and anomalies

The graphic page

- is the **drawing surface** on which R renders plots. Think of it as a blank canvas associated with a graphics device (screen window, PDF, PNG file, etc.).
- One **device** → can contain **multiple pages**
- One **page** → can contain **one or multiple plots** (depending on layout)
- **Graphics device**: where output goes
  - Examples: screen (RStudio plot pane), pdf(), png()
- **Graphics page**: a single “frame” within that device
- Analogy:
  - Device = a notebook
  - Page = one sheet inside the notebook

The graphic page - II

- Each new high-level plotting function (like plot(), hist(), boxplot()) **starts a new graphics page**:

```
plot(1:10)    # Page 1
plot(10:1)    # Page 2 (previous plot replaced in viewer)
```

- In interactive devices (e.g., RStudio), you *see only one page at a time*, but you can navigate history.

- **2. Forcing a new page**
- You can explicitly start a new page:

```
plot.new()
```

- This clears the current page and prepares a blank canvas.

The graphic page

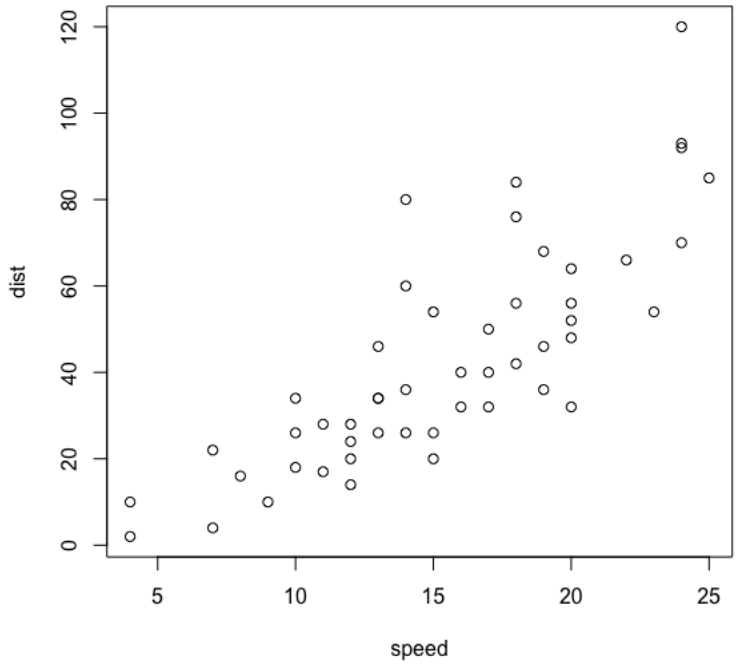
- Graphic functions draw in a new page
- main functions
  - **`plot`**
  - **`hist`**
  - **`barplot`**
  - **`boxplot`**
  - **`pie`**
  - …
- additional components add element to the current pa
  - **`lines`**
  - **`abline`**
  - **`text`**
  - **`axis`**
  - …

In the following, with "plot" in standard character we will mean a generig graphic function, while with **`plot`** we will mean the specific function with that name

Standard components and default values

- plot functions have a set of standard components and default values
  - in many cases default values are drawn from data, e.g. column names in data frames
- graphic appearance can be highly personalised
- by default, if the parameter is a two-columns dataframe we obtain a scatter plot of the columns

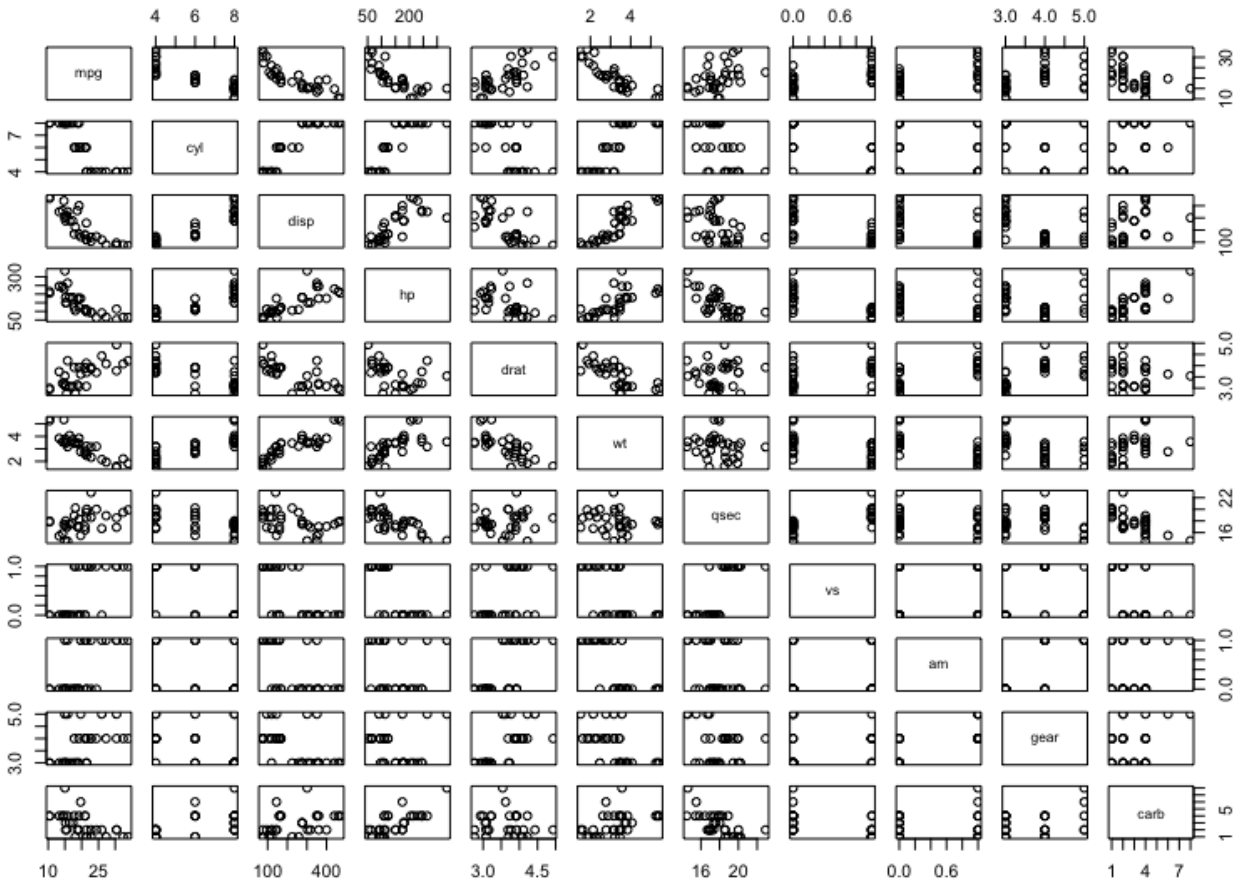
```
# plot data from the predefined dataframe cars
# a dataframe with two columns (dist,speed)
# Speed and Stopping Distances of Cars (ethz.ch)
plot(cars)
```



Standard components and default values

```
# plot data from the
# predefined
# dataframe mtcars
plot(mtcars)
```

- by default, if the parameter is a many-columns dataframe we obtain the *pair-plot*
  - scatter plot of all the pairs
- [mtcars]

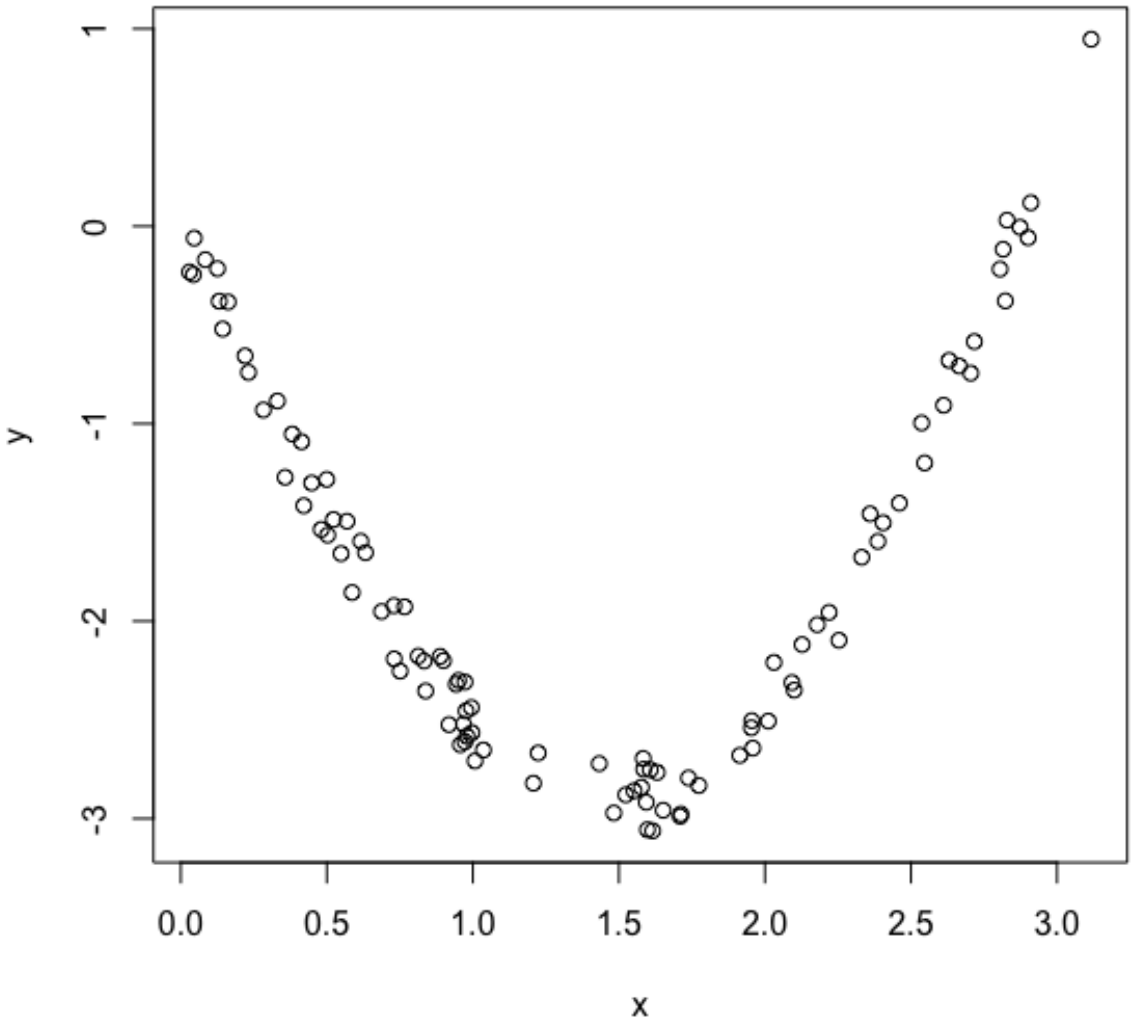


Standard components (non-exhaustive)

- x label, ticks, values, ... (same for y)
- types
  - “p”: Points (default)
  - “l”: Lines
  - “b”: Both
  - “c”: The lines part alone of “b”
  - “o”: Both “overplotted”
  - “h”: **Histogram** like (or high-density) vertical lines.
  - “n”: No plotting.

Plot output type

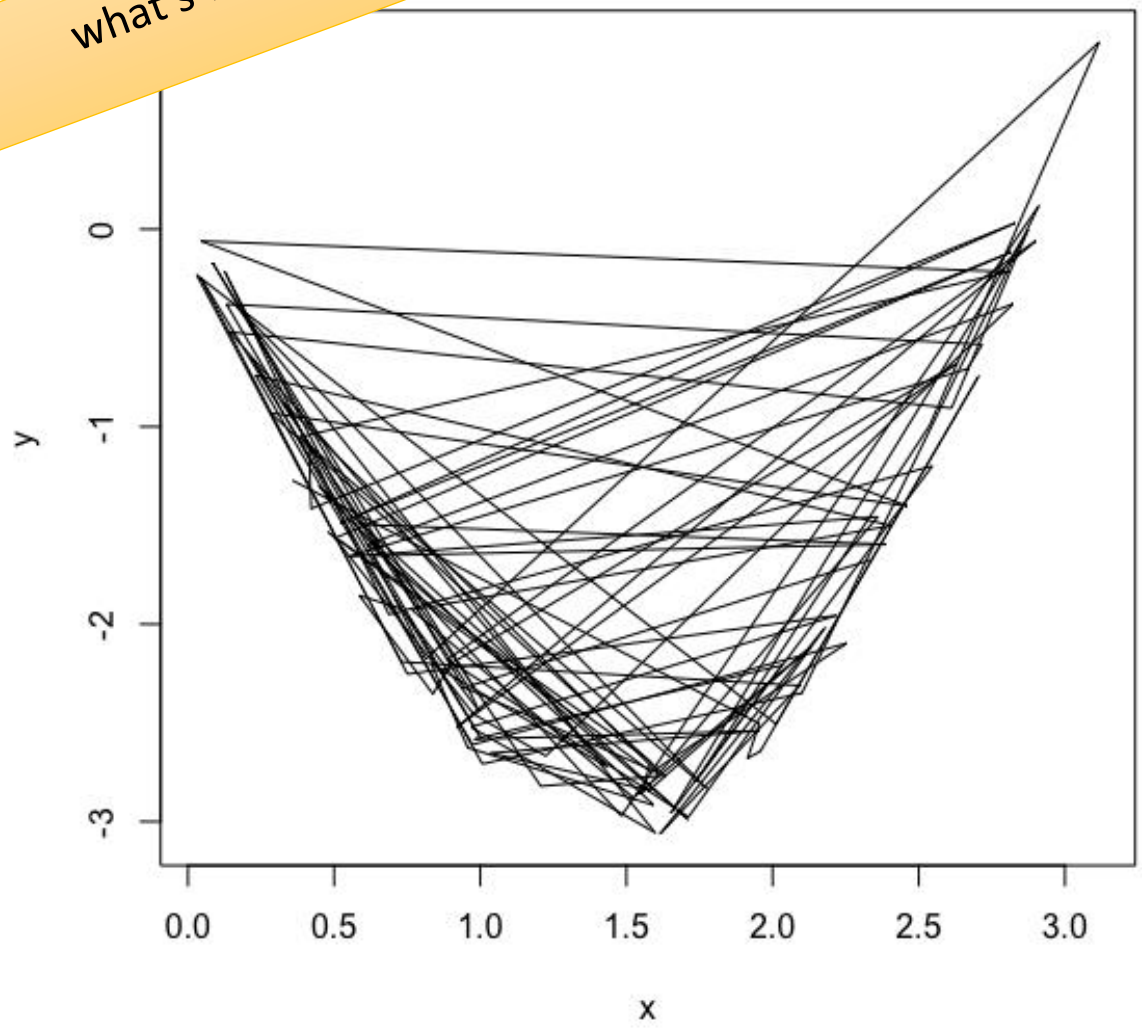
```
rm(list = ls())
fun <- function(x){
  noise <- 0.2
  y <- 0.1*x^2-pi*sin(x)
    + noise * runif(n,-1,1)
}
n <- 100
seed <- 1234
set.seed(seed)
x <- runif(n,0,pi)
y <- fun(x)
plot(x,y)
```



Plot output type

```
rm(list = ls())
fun <- function(x){
  noise <- 0.2
  y <- 0.1*x^2-pi*sin(x)
    + noise * runif(n,-1,1)
}
n <- 100
seed <- 1234
set.seed(seed)
x <- runif(n,0,pi)
y <- fun(x)
plot(x,y,type = "l")
```

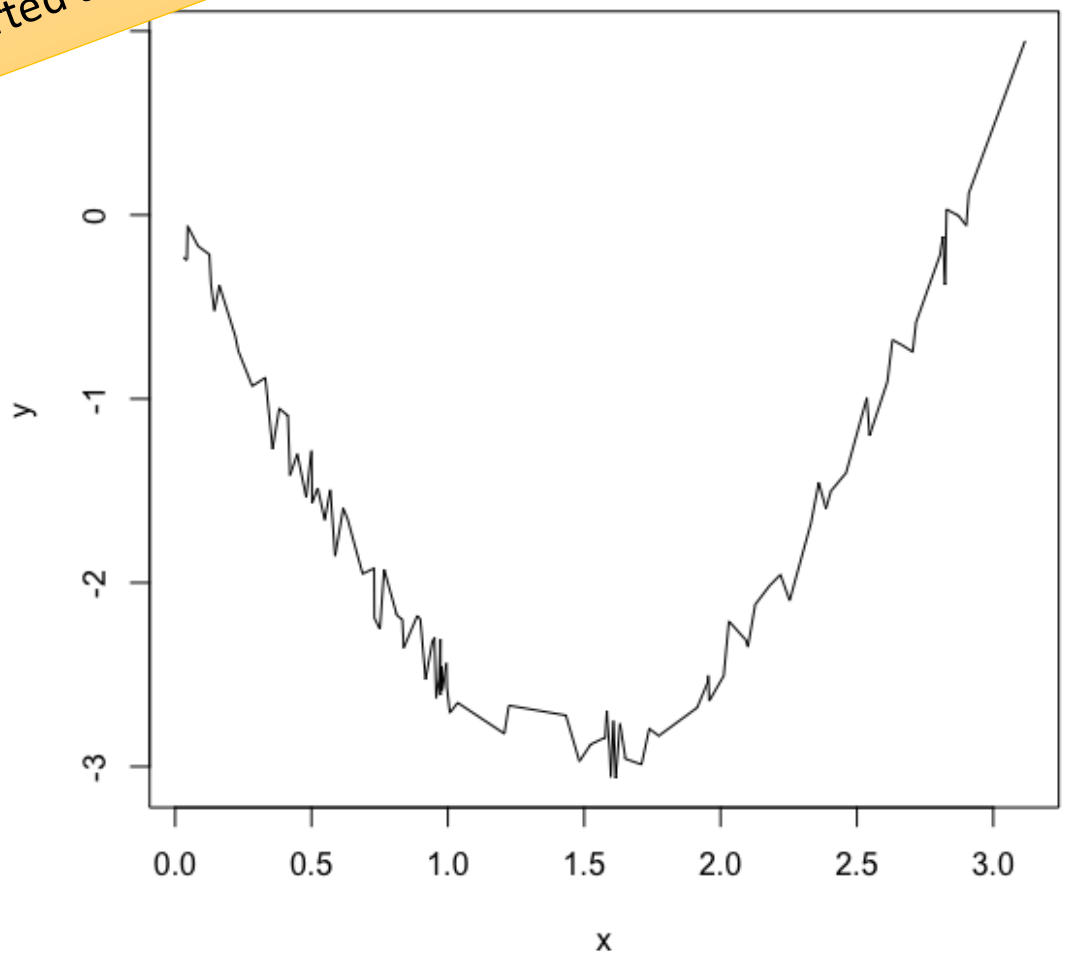
what's wrong?



Plot output type

plotting lines requires the
x data to be ***sorted***
y data must be sorted accordingly

```
...
x <- runif(n,0,pi)
y <- fun(x)
o_x<- order(x)
   # the index sequence that makes x
sorted
x <- x[o_x]
   # reuse the same variables
y <- y[o_x]
plot(x,y, type = "l")
```



Another plotting library: **`ggplot2`**

- data + aesthetics + geometry
- Layered approach
- Declarative syntax
  - you specify what you want, not a sequence of component

see plotting:R_plotting_commented.R